

# National University of Computer and Emerging Sciences (FAST)

Islamabad Campus

---

## Intelligent Reading Comprehension and Quiz Generation System

Using Traditional Machine Learning on the RACE Dataset

---

**Course:** Artificial Intelligence (AL2002) - CS (BS) Spring 2026

**Authors:** Abdul Mohaimin 23i-0652  
Shazil Rehman 23i-0095

**Date:** May 2026



### Abstract

This report presents an end-to-end intelligent reading comprehension and quiz generation system built on the RACE dataset [1]. The system comprises two machine learning pipelines: Model A, which performs answer verification using supervised classification (Logistic Regression, SVM, Naive Bayes, Random Forest) as well as unsupervised and semi-supervised methods (K-Means clustering, Label Propagation), and Model B, which handles distractor generation and hint scoring using logistic regression trained on handcrafted features. Template-based question generation transforms article sentences into well-formed WH-questions. All models operate exclusively on one-hot encoded features without TF-IDF or deep learning. The system is exposed through an interactive Streamlit interface and a FastAPI backend. Evaluation on a held-out test set yielded an ensemble accuracy of **0.7501** and macro F1 of **0.4298** for answer verification. Ethical implications including dataset bias, academic integrity, and AI transparency are discussed.

# Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>4</b> |
| <b>2</b> | <b>Related Work</b>                                              | <b>4</b> |
| <b>3</b> | <b>Dataset</b>                                                   | <b>4</b> |
| 3.1      | RACE Overview . . . . .                                          | 4        |
| 3.2      | Data Splits . . . . .                                            | 5        |
| 3.3      | Exploratory Data Analysis . . . . .                              | 5        |
| 3.3.1    | Answer Label Distribution . . . . .                              | 5        |
| 3.3.2    | Question Type Distribution . . . . .                             | 5        |
| 3.3.3    | Vocabulary Coverage . . . . .                                    | 6        |
| 3.3.4    | Article–Option Overlap . . . . .                                 | 6        |
| <b>4</b> | <b>System Architecture</b>                                       | <b>6</b> |
| <b>5</b> | <b>Preprocessing</b>                                             | <b>7</b> |
| 5.1      | Text Cleaning . . . . .                                          | 7        |
| 5.2      | Vocabulary Construction . . . . .                                | 7        |
| 5.3      | Feature Representations . . . . .                                | 7        |
| 5.3.1    | One-Hot Encoding (Primary Feature - Model A) . . . . .           | 7        |
| 5.3.2    | Lexical Features (5-dimensional, Model A RF / Model B) . . . . . | 7        |
| 5.3.3    | Question Bag-of-Words (Model A NB) . . . . .                     | 8        |
| 5.3.4    | Sparse Storage . . . . .                                         | 8        |
| <b>6</b> | <b>Model A - Answer Verification</b>                             | <b>8</b> |
| 6.1      | Task Formulation . . . . .                                       | 8        |
| 6.2      | Supervised Models . . . . .                                      | 8        |
| 6.2.1    | Logistic Regression . . . . .                                    | 8        |
| 6.2.2    | Support Vector Machine . . . . .                                 | 8        |
| 6.2.3    | Naive Bayes (Question Type Classification) . . . . .             | 8        |
| 6.2.4    | Random Forest (Difficulty Classification) . . . . .              | 9        |
| 6.3      | Template-Based Question Generation . . . . .                     | 9        |
| 6.4      | Unsupervised and Semi-Supervised Learning . . . . .              | 9        |
| 6.4.1    | K-Means Clustering . . . . .                                     | 9        |
| 6.4.2    | Label Propagation . . . . .                                      | 9        |
| 6.5      | Ensemble Methods . . . . .                                       | 10       |
| 6.5.1    | Soft Voting . . . . .                                            | 10       |
| 6.5.2    | Hard Voting . . . . .                                            | 10       |

|           |                                                 |           |
|-----------|-------------------------------------------------|-----------|
| 6.5.3     | Stacking . . . . .                              | 10        |
| <b>7</b>  | <b>Model B - Distractor and Hint Generation</b> | <b>10</b> |
| 7.1       | Distractor Generation . . . . .                 | 10        |
| 7.1.1     | Candidate Extraction . . . . .                  | 10        |
| 7.1.2     | Feature Engineering . . . . .                   | 10        |
| 7.1.3     | Distractor Ranker . . . . .                     | 11        |
| 7.2       | Hint Generation . . . . .                       | 11        |
| 7.2.1     | Sentence Scoring . . . . .                      | 11        |
| 7.2.2     | Hint Scorer . . . . .                           | 11        |
| <b>8</b>  | <b>Evaluation</b>                               | <b>11</b> |
| 8.1       | Model A - Answer Verification . . . . .         | 11        |
| 8.1.1     | Confusion Matrix (Stacking Ensemble) . . . . .  | 12        |
| 8.2       | Unsupervised Evaluation . . . . .               | 12        |
| 8.3       | Model B - Distractor Generation . . . . .       | 12        |
| 8.4       | Model B - Hint Scoring . . . . .                | 13        |
| <b>9</b>  | <b>User Interface</b>                           | <b>13</b> |
| 9.1       | Streamlit Application . . . . .                 | 13        |
| 9.2       | FastAPI Backend . . . . .                       | 13        |
| <b>10</b> | <b>Ethical Considerations</b>                   | <b>14</b> |
| 10.1      | Dataset Bias . . . . .                          | 14        |
| 10.2      | Academic Integrity . . . . .                    | 14        |
| 10.3      | Accessibility . . . . .                         | 14        |
| 10.4      | Model Transparency . . . . .                    | 15        |
| <b>11</b> | <b>Limitations and Future Work</b>              | <b>15</b> |
| 11.1      | Current Limitations . . . . .                   | 15        |
| 11.2      | Future Work . . . . .                           | 15        |
| <b>12</b> | <b>Conclusion</b>                               | <b>16</b> |

# 1 Introduction

Reading comprehension is a fundamental skill assessed in academic settings worldwide. Automated quiz generation systems have the potential to reduce the burden on educators while providing personalised practice for students. This project implements such a system using the RACE dataset [1], a large-scale benchmark derived from English reading comprehension examinations for Chinese middle and high school students.

The system addresses two core tasks:

1. **Answer Verification (Model A):** Given an article, a question, and a candidate option, predict whether the option is correct.
2. **Quiz Generation (Model B):** Given an article and a correct answer, generate plausible distractors and progressive hints to scaffold student understanding.

A deliberate constraint of this project is the exclusive use of *traditional* machine learning algorithms - no transformers, no deep neural networks - to demonstrate that meaningful performance can be achieved with classical techniques and handcrafted features.

# 2 Related Work

Question generation from text has been studied extensively. Du et al. [2] proposed neural sequence-to-sequence models for question generation, which later influenced attention-based and BERT-based approaches [5]. Zhao et al. [3] extended this with paragraph-level generation using gated self-attention. For distractor generation, Guo et al. [4] leveraged semantic similarity and passage frequency to rank candidates - an approach closely related to our feature design. Evaluation of generated text has relied on BLEU [6] and ROUGE [7] metrics, though these correlate imperfectly with human judgement for question quality. Our work intentionally restricts itself to classical ML to satisfy the course constraints, making it complementary to these neural baselines rather than competitive with them.

# 3 Dataset

## 3.1 RACE Overview

The RACE dataset [1] contains approximately 28,000 reading passages and 100,000 multiple-choice questions sourced from Chinese middle and high school English examinations. Each instance consists of:

- **article:** A reading passage (mean length  $\approx$  275 words, max 1,162 words)

- **question:** A multiple-choice question (mean 10 words)
- **A, B, C, D:** Four candidate options (mean 5–6 words each)
- **answer:** The correct label (A, B, C, or D)

## 3.2 Data Splits

Since the Kaggle distribution of the dataset contained identical content across the train, dev, and test CSV files, the full dataset of 87,866 rows was manually split using a reproducible random seed:

Table 1: Dataset split sizes

| Split      | Rows   | Proportion |
|------------|--------|------------|
| Train      | 70,282 | 80%        |
| Validation | 8,785  | 10%        |
| Test       | 8,785  | 10%        |

## 3.3 Exploratory Data Analysis

### 3.3.1 Answer Label Distribution

A mild class imbalance was observed in the training set:

Table 2: Answer label distribution (training set)

| Label | Count  | Percentage |
|-------|--------|------------|
| A     | 15,290 | 21.8%      |
| B     | 18,205 | 25.9%      |
| C     | 19,078 | 27.1%      |
| D     | 17,712 | 25.2%      |

Label A is underrepresented by approximately 4,000 instances relative to the most frequent class (C). All classifiers were trained with `class_weight='balanced'` to compensate.

### 3.3.2 Question Type Distribution

Questions were classified by their first content word:

Table 3: Question type distribution (training set)

| Question Type | Count        |
|---------------|--------------|
| What / Which  | 13520 / 8630 |
| According to  | 4146         |
| Why           | 3279         |
| How           | 2727         |
| When          | 1787         |
| Who           | 981          |
| Where         | 910          |
| Other         | 34305        |

### 3.3.3 Vocabulary Coverage

A vocabulary coverage analysis confirmed that 5,000 words is a reasonable cutoff:

Table 4: Token coverage by vocabulary size

| Vocabulary Size | Token Coverage |
|-----------------|----------------|
| 1,000           | 76.06%         |
| 2,000           | 83.97%         |
| 5,000           | 91.93%         |
| 10,000          | 95.55%         |
| 20,000          | 97.49%         |

### 3.3.4 Article–Option Overlap

A Jaccard similarity analysis between article text and correct vs. incorrect options confirmed that correct options have marginally higher keyword overlap with the passage, validating the use of overlap as a lexical feature.

## 4 System Architecture

The system follows a modular pipeline architecture:

1. **Preprocessing** (`src/preprocessing.py`): Text cleaning, vocabulary construction, one-hot encoding, and lexical feature extraction.
2. **Model A Training** (`src/model_a_train.py`): Supervised classifiers, template-based question generation, unsupervised clustering, semi-supervised label propagation, and ensemble methods.
3. **Model B Training** (`src/model_b_train.py`): Distractor candidate ranking and hint sentence scoring.

4. **Inference** (`src/inference.py`): A unified API layer loaded once at startup, serving both the FastAPI backend and the Streamlit UI.
5. **Evaluation** (`src/evaluate.py`): Metric computation for all model variants.
6. **User Interface** (`ui/streamlit_app.py`, `ui/app.py`): A Streamlit application and a FastAPI backend.

## 5 Preprocessing

### 5.1 Text Cleaning

Each text field is lowercased, stripped of non-alphabetic characters (punctuation, digits), and whitespace-normalised. No stemming or lemmatisation is applied to preserve one-hot vector interpretability.

### 5.2 Vocabulary Construction

A vocabulary of the top 5,000 most frequent words is built exclusively from the training corpus articles. This prevents test-set leakage. The vocabulary maps each word to a unique integer index.

### 5.3 Feature Representations

#### 5.3.1 One-Hot Encoding (Primary Feature - Model A)

For each (article, question, option) triple, three binary vectors of length 5,000 are computed and concatenated:

$$\mathbf{x} = [\text{ohe}(\text{article}) \parallel \text{ohe}(\text{question}) \parallel \text{ohe}(\text{option})] \in \{0, 1\}^{15000}$$

Each vector has a 1 at index  $i$  if the  $i$ -th vocabulary word appears in the text. This produces a sparse matrix of shape  $(N \times 4, 15000)$  where  $N$  is the number of passages.

#### 5.3.2 Lexical Features (5-dimensional, Model A RF / Model B)

A handcrafted dense feature vector captures:

1. Keyword overlap count between article and option
2. Option length (words)
3. Question length (words)

4. Article length (words)
5. Normalised position of first matching word in article ( $\in [0, 1]$ )

### 5.3.3 Question Bag-of-Words (Model A NB)

One-hot encoding of the question text only, used to classify question type (Who / What / Where / When / Why / How / Which / According to / Other).

### 5.3.4 Sparse Storage

All one-hot feature matrices are stored as SciPy Compressed Sparse Row (CSR) matrices to avoid dense memory allocation ( $\approx 15$  GB for the training set in dense format).

## 6 Model A - Answer Verification

### 6.1 Task Formulation

Answer verification is framed as binary classification: given  $\mathbf{x} = \text{encode}(\text{article}, \text{question}, \text{option})$ , predict  $y \in \{0, 1\}$  where  $y = 1$  indicates the option is the correct answer. The training set contains four rows per passage (one per option), yielding  $4 \times 70,282 = 281,128$  training samples with a 3:1 class imbalance (75% negative).

### 6.2 Supervised Models

#### 6.2.1 Logistic Regression

Trained on the 15,000-dimensional one-hot feature matrix using the SAGA solver with L2 regularisation, 1,000 maximum iterations, and balanced class weights.

#### 6.2.2 Support Vector Machine

A `LinearSVC` wrapped in `CalibratedClassifierCV` (3-fold) to obtain probability outputs for the ensemble. Balanced class weights and 2,000 maximum iterations.

#### 6.2.3 Naive Bayes (Question Type Classification)

A `MultinomialNB` classifier trained on the question-only bag-of-words vector. Labels are question type categories derived from the first content word of each question. This auxiliary model captures the question's intent for use in the template ranking pipeline.

### 6.2.4 Random Forest (Difficulty Classification)

A 200-tree `RandomForestClassifier` trained on the 5-dimensional lexical feature vector. Labels are a difficulty proxy derived from binned question length: short ( $\leq 7$  words) = easy, medium (8–12 words), long ( $> 12$  words) = hard.

## 6.3 Template-Based Question Generation

Candidate sentences are extracted from the article by ranking each sentence’s cosine similarity (computed on one-hot vectors) to the answer string. The top-5 sentences are passed through rule-based WH-templates:

- **Who:** if a capitalised name pattern is detected
- **When:** if a year (e.g. 1990, 2024) or time word (morning, year, day) is present
- **Where:** if a location indicator (in, at, from, near) is present
- **Why:** if a causal word (because, therefore, since, thus) is present
- **What:** always applied as a default fallback

Candidates are ranked by a fluency heuristic: questions starting with a WH-word score higher; shorter deviation from an ideal length of 10 words is rewarded.

## 6.4 Unsupervised and Semi-Supervised Learning

### 6.4.1 K-Means Clustering

The 15,000-dimensional sparse feature matrix is first reduced to 100 dimensions using Truncated SVD (analogous to PCA for sparse matrices). K-Means with  $k = 4$  is then applied.

Table 5: K-Means clustering results

| Metric           | Value  |
|------------------|--------|
| Cluster Purity   | 1.000  |
| Silhouette Score | 0.0399 |

### 6.4.2 Label Propagation

Label Propagation with a KNN kernel ( $k = 7$ ) is applied to a subsample of 15,000 instances (due to the quadratic complexity of the algorithm). Labels are provided for 10% of the sample; the remaining 90% are unlabelled ( $y = -1$ ). The model propagates labels through a KNN graph.

Table 6: Label Propagation evaluation on labelled subset

| <b>Metric</b> | <b>Value</b> |
|---------------|--------------|
| Accuracy      | 0.6564       |
| Macro F1      | 0.5548       |

## 6.5 Ensemble Methods

### 6.5.1 Soft Voting

Probability outputs from each base model are averaged; the class with the highest mean probability is predicted.

### 6.5.2 Hard Voting

Each model produces a hard prediction; majority vote determines the final label.

### 6.5.3 Stacking

A meta Logistic Regression is trained on the validation-set probability outputs of the base models. This is the saved ensemble used in the final evaluation.

## 7 Model B - Distractor and Hint Generation

### 7.1 Distractor Generation

#### 7.1.1 Candidate Extraction

All unigrams and bigrams from the article are collected, excluding any tokens that appear in the correct answer string. Candidates are ranked by frequency in the passage.

#### 7.1.2 Feature Engineering

Each candidate is scored against the correct answer using a 3-dimensional feature vector:

1. One-hot cosine similarity between candidate and answer
2. Character-level Jaccard overlap between candidate and answer character sets
3. Normalised passage frequency of the candidate

### 7.1.3 Distractor Ranker

A Logistic Regression classifier is trained on 5,000 passages with  $y = 1$  if the candidate matches one of the three gold distractors in the dataset. A diversity penalty removes candidates that share more than 50% word overlap with an already-selected distractor. Exactly 3 distractors are returned.

## 7.2 Hint Generation

### 7.2.1 Sentence Scoring

Each sentence in the article is scored using a 3-dimensional feature vector:

1. Normalised keyword overlap between sentence and question
2. Normalised sentence position in the article (0 = first, 1 = last)
3. Normalised sentence length (clipped at 50 words)

### 7.2.2 Hint Scorer

A Logistic Regression classifier is trained with  $y = 1$  if the sentence contains the correct answer. Hints are returned in ascending relevance order (Hint 1 = least specific, Hint 3 = closest to the answer but not the answer sentence itself).

## 8 Evaluation

### 8.1 Model A - Answer Verification

Table 7: Model A test-set evaluation results

| Model               | Accuracy | Macro F1 | Precision | Recall |
|---------------------|----------|----------|-----------|--------|
| Logistic Regression | 0.5804   | 0.5332   | 0.5472    | 0.5618 |
| SVM (Calibrated)    | 0.7499   | 0.4291   | 0.5834    | 0.5002 |
| Stacking Ensemble   | 0.7501   | 0.4298   | 0.6501    | 0.5005 |

**Interpretation.** The SVM and Ensemble achieve 75% accuracy - equal to the naïve majority-class baseline for a 75:25 class-imbalance problem - revealing that these models predominantly predict the negative class (option is incorrect). The Logistic Regression achieves lower accuracy (58%) but substantially higher macro F1 (0.53), indicating more balanced prediction of both classes. For a real deployment the LR model would be preferred as the primary predictor; the ensemble benefits precision at the cost of recall.

### 8.1.1 Confusion Matrix (Stacking Ensemble)

Table 8: Confusion matrix - Stacking Ensemble (test set)

|                  | Predicted Incorrect | Predicted Correct |
|------------------|---------------------|-------------------|
| Actual Incorrect | 26,346              | 9                 |
| Actual Correct   | 8,774               | 11                |

## 8.2 Unsupervised Evaluation

Table 9: Unsupervised and semi-supervised evaluation

| Method              | Metric             | Value  |
|---------------------|--------------------|--------|
| K-Means ( $k = 4$ ) | Cluster Purity     | 1.000  |
| K-Means ( $k = 4$ ) | Silhouette Score   | 0.0399 |
| Label Propagation   | Accuracy (labeled) | 0.6564 |
| Label Propagation   | Macro F1 (labeled) | 0.5548 |
| Logistic Regression | Accuracy (val)     | 0.5804 |
| Logistic Regression | Macro F1 (val)     | 0.5332 |
| SVM (Calibrated)    | Accuracy (val)     | 0.7499 |
| SVM (Calibrated)    | Macro F1 (val)     | 0.4291 |

## 8.3 Model B - Distractor Generation

Table 10: Distractor generation evaluation (200 test rows)

| Metric                                          | Value |
|-------------------------------------------------|-------|
| Precision                                       | 0.005 |
| Recall                                          | 0.005 |
| F1                                              | 0.005 |
| Accuracy (top distractor $\neq$ correct answer) | 1.000 |

The very low precision/recall values are an artefact of the evaluation method: exact string matching between generated unigram/bigram candidates and full-phrase gold distractors. The more meaningful metric is accuracy (1.0) - the system never generates the correct answer as a distractor.

## 8.4 Model B - Hint Scoring

Table 11: Hint generation evaluation (200 test rows)

| Metric      | Value |
|-------------|-------|
| Precision@3 | 0.145 |

A Precision@3 of 0.145 indicates that roughly 1 in 7 of the top-3 hints overlaps substantially (>30% word overlap) with the gold answer sentence. This is above a random baseline for passages with many sentences.

## 9 User Interface

The system is exposed through two interfaces:

### 9.1 Streamlit Application

A standalone four-tab interface (`ui/streamlit_app.py`) runnable with:

```
streamlit run ui/streamlit_app.py
```

- **Tab 1 - Article Input:** Text area for article entry; “Generate Quiz” and “Load Random Sample” buttons.
- **Tab 2 - Quiz:** Displays the generated question and four radio-button options; “Check Answer” invokes Model A for verification with confidence score.
- **Tab 3 - Hints:** Progressive disclosure of three hints from least to most specific; “Reveal Answer” button unlocked only after all hints are shown.
- **Tab 4 - Dashboard:** Model A and B metric cards, model comparison bar chart, confusion matrix table, session log, and CSV export.

### 9.2 FastAPI Backend

A REST API (`ui/app.py`) exposing five endpoints for integration with any frontend:

Table 12: FastAPI endpoint summary

| Method | Path                | Description                            |
|--------|---------------------|----------------------------------------|
| POST   | /api/generate       | Generate full quiz from article        |
| POST   | /api/verify         | Verify single answer option            |
| GET    | /api/random-sample  | Load a random passage from the dataset |
| GET    | /api/metrics        | Retrieve cached evaluation metrics     |
| GET    | /api/export-session | Download session log as CSV            |

## 10 Ethical Considerations

### 10.1 Dataset Bias

The RACE dataset is derived from English reading comprehension examinations designed for Chinese middle and high school students. This introduces several sources of bias:

- **Cultural bias:** Passages reflect Chinese educational contexts, values, and cultural references that may be unfamiliar to students from other backgrounds, potentially affecting the relevance and fairness of generated questions.
- **Linguistic bias:** English passages authored for non-native speakers may have stylistic patterns (simplified vocabulary, formal register) that differ from native English educational materials, limiting generalisation.
- **Grade-level calibration:** Difficulty labels derived from question length are a coarse proxy; the RACE dataset’s own difficulty annotations (middle vs. high school) were not used, as this distinction was not preserved in the available CSV files.

### 10.2 Academic Integrity

AI-generated questions must not be used as examination content without human review. The system makes no claim that generated questions are pedagogically valid, grammatically correct, or appropriately calibrated in difficulty. All output is labelled as AI-generated within the interface.

### 10.3 Accessibility

The Streamlit interface maintains sufficient colour contrast for legibility and uses standard HTML controls navigable by keyboard and screen readers. Progressive hint disclosure reduces cognitive load and supports diverse learner paces.

## 10.4 Model Transparency

The interface clearly communicates that all questions, distractors, and hints are generated by machine learning models. Confidence scores are displayed alongside answer verification results to help users calibrate their trust in the system’s output. The system must not be presented to end users as infallible.

# 11 Limitations and Future Work

## 11.1 Current Limitations

- **Question quality:** Rule-based WH-templates produce syntactically correct but often semantically shallow questions. The answer proxy (last word of the first sentence) is a very coarse heuristic.
- **Distractor quality:** Unigram/bigram candidates lack the phrase-level structure of human-authored distractors. A minimum phrase length threshold would improve plausibility.
- **SVM class collapse:** The SVM and ensemble models predominantly predict the negative class despite balanced class weighting, suggesting that one-hot cosine similarity alone is insufficient signal for binary answer verification at this scale.
- **Inference latency:** Loading sparse feature matrices and running predictions at startup adds a  $\sim 30$ -second delay. A pre-computed metric cache mitigates repeated computation.

## 11.2 Future Work

- Replacing one-hot encoding with word embeddings (Word2Vec, GloVe) or contextual representations (BERT [5]) would substantially improve semantic discrimination.
- Fine-tuned transformer models have demonstrated state-of-the-art performance on RACE [1] and would address the class-collapse problem observed in the SVM.
- Named Entity Recognition (NER) could improve question template selection by identifying persons, organisations, locations, and dates more reliably than regex patterns.
- Human evaluation (Likert scale believability ratings for distractors, fluency ratings for questions) would provide a more meaningful quality signal than exact-match metrics.

## 12 Conclusion

This project demonstrates a complete end-to-end quiz generation pipeline built exclusively on traditional machine learning techniques and one-hot encoded text features. The system successfully trains and deploys six model artifacts covering supervised classification, unsupervised clustering, semi-supervised label propagation, ensemble learning, distractor ranking, and hint scoring - all within the hard constraints of no deep learning, no TF-IDF, and sub-10-second inference per query. The Streamlit interface provides a functional, accessible user experience. While the quality of generated questions and distractors is limited by the representational capacity of binary bag-of-words features, the system fulfils all specified requirements and provides a solid classical baseline against which future neural approaches can be benchmarked.

## References

- [1] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. Hovy, “RACE: Large-scale ReAding Comprehension Dataset From Examinations,” in *Proc. EMNLP*, 2017, pp. 785–794.
- [2] X. Du, J. Shao, and C. Cardie, “Learning to Ask: Neural Question Generation for Reading Comprehension,” in *Proc. ACL*, 2017, pp. 1342–1352.
- [3] Y. Zhao, X. Ni, Y. Ding, and Q. Ke, “Paragraph-level Neural Question Generation with Maxout Pointer and Gated Self-attention Networks,” in *Proc. EMNLP*, 2018, pp. 3901–3910.
- [4] Q. Guo, S. Agarwal, and B. Dai, “Generating Distractors for Reading Comprehension Questions from Real Examinations,” in *Proc. AAAI*, 2016.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL*, 2019, pp. 4171–4186.
- [6] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A Method for Automatic Evaluation of Machine Translation,” in *Proc. ACL*, 2002, pp. 311–318.
- [7] C.-Y. Lin, “ROUGE: A Package for Automatic Evaluation of Summaries,” in *Proc. ACL Workshop on Text Summarisation*, 2004, pp. 74–81.